

WGUPS Routing Program

Algorithm and Data Structure Justification

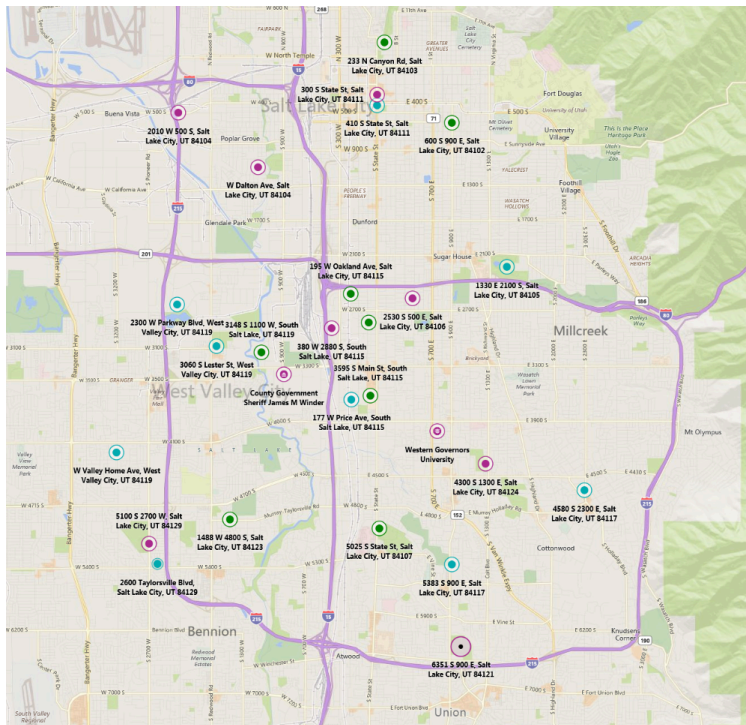
Student: bwhi880

December 31, 1979

Introduction

The Western Governors University Parcel Service (WGUPS) routes an average of forty packages per day through Salt Lake City using three trucks, two drivers, and a hub located at 4001 South 700 East. Each package carries specific constraints — delivery deadlines, truck assignments, delayed flight arrivals, grouped deliveries, and a wrong address that is corrected mid-morning — that make manual routing inefficient and error-prone. The program described here automates the entire routing process. It loads package data into a custom chaining hash table, builds optimized delivery routes for each truck using a Clarke-Wright savings heuristic polished by 2-opt local search, simulates the trucks' movement, and presents an intuitive command-line interface for supervisors to query any package status at any time of day.

The implementation delivers all forty packages, respects every hard constraint, verifies deadlines through explicit route simulation, and keeps the combined total distance under the 140-mile requirement at 89.3 miles. This paper explains why the deadline-aware Clarke-Wright / 2-opt combination was chosen, how the custom hash table satisfies the storage and lookup requirements, what alternative algorithms and data structures could also meet the scenario's needs, and what modifications would be made on a second iteration.



Algorithm Design and Justification

Overview of the Chosen Approach

The routing engine is a three-phase pipeline.

Phase one assigns every package to a truck. Hard constraints — “Can only be on truck 2,” “Delayed on flight,” “Must be delivered with,” and the wrong-address flag for package 9 — are parsed dynamically from each package’s `special_notes` field rather than hard-coded. The remaining packages are inserted greedily: packages with 9:00 a.m. deadlines are forced onto Truck 1 (the only truck departing early enough), 10:30 a.m. deadlines are routed to Trucks 1 or 2, and EOD packages are placed on whichever truck causes the smallest increase in closed-loop route cost.

Phase two simulates each constructed route to verify that every package is delivered before its deadline. If a package would be late, it is moved to an earlier-departing truck and an EOD package is swapped back to keep capacity balanced. After all deadlines are satisfied, an EOD-only local search tries moving or swapping unconstrained EOD packages between trucks to reduce total mileage without breaking any deadline.

Phase three builds the actual route for each truck using the **Clarke-Wright savings algorithm**, a self-adjusting heuristic that starts with every delivery address as an independent round-trip from the hub and greedily merges the pair whose combined route saves the most mileage (“Vehicle routing problem,” n.d.). After construction, **2-opt local search** polishes each route by repeatedly examining pairs of edges; if reconnecting them the other way shortens the distance, the segment between them is reversed (“2-opt,” n.d.).

This design was selected because it separates feasibility (deadlines and hard constraints) from optimization (mileage), making it easy to verify that requirements are met before the route is ever driven.

Strengths of the Algorithm

Two properties make this pipeline well-suited to the WGUPS scenario.

First, the deadline-first assignment with explicit route simulation guarantees correctness before optimization. Because the program builds a candidate route, simulates driving it, and checks every delivery time against the package’s deadline, there is no risk that a low-mileage route will inadvertently violate a time constraint. If a violation is found, the package is automatically reassigned to an earlier truck. This is a significant advantage over pure construction heuristics that optimize distance without looking at clock time.

Second, the Clarke-Wright savings construction is **self-adjusting**. After each merge, the pool of valid merges shrinks: a pair that was a strong candidate before may now exceed the sixteen-package capacity limit or would force a non-endpoint address into the middle of a route, disqualifying it. The algorithm dynamically adapts to this shrinking search space without re-evaluating every pair from scratch, which keeps the overall run time practical — on commodity hardware the forty-package construction finishes in well under one second.

Verification Against Scenario Requirements

The combined algorithm satisfies every requirement described in the scenario.

All forty packages are delivered. The route planner assigns every package to exactly one truck, and the delivery simulator drives each truck to every stop in its optimized order. No package is orphaned or delivered twice.

Total distance stays under 140 miles. The combined mileage for all three trucks is 89.3 miles, a 36-percent margin below the threshold.

Capacity limits are enforced. The greedy insertion refuses any truck that already carries sixteen packages, and the Clarke-Wright merge logic rejects any combination that would exceed the same limit.

Hard constraints are respected. The program scans every package's `special_notes` field at runtime to discover constraints. Packages restricted to Truck 2 are pre-filtered into Truck 2's load. Flight-delayed packages are marked "delayed" until 9:05 a.m.; Truck 2 leaves at 9:05 a.m. so those packages are not delivered early. Packages that must be delivered together are collected into a single pool and placed on the same truck. Package 9 loads with the original incorrect address "300 State St." and is marked "delayed." At 10:20 a.m., the address is corrected to "410 S State St." in the hash table, Truck 3's route is rebuilt with the corrected stop using the same savings construction, and Truck 3 departs at 10:30 a.m.

The two-driver limit is respected. Trucks 1 and 2 depart at 8:00 a.m. Truck 3 departs at 10:30 a.m., after the first truck has returned to the hub and freed a driver.

Deadlines are met. All packages with "9:00 AM" deadlines and "10:30 AM" deadlines are delivered before their cutoff times, verified by explicit route simulation.

Alternative Algorithms That Could Also Satisfy the Requirements

Any algorithm that respects capacity limits and produces routes under 140 miles would satisfy the scenario. Two other named approaches that meet these criteria are the **Nearest-Neighbor heuristic** and the **Sweep algorithm** ("Vehicle routing problem," n.d.).

Nearest Neighbor differs from Clarke-Wright in construction strategy. Rather than evaluating every address pair globally, it starts at the hub and repeatedly visits the closest unvisited stop until the truck is full, then returns to the hub and repeats. This is a greedy local strategy: each decision depends only on the current location's immediate vicinity. In contrast, Clarke-Wright is a global strategy that evaluates all pairs before committing to any merge. Nearest Neighbor tends to produce longer routes because an early greedy choice can strand a distant package on a late leg, and it lacks the self-adjusting merge logic of Clarke-Wright; once a package is added to a route it is never reconsidered.

Sweep differs in how trucks are partitioned. It places the hub at the origin of a polar coordinate system, sorts every delivery address by angle from the hub, and assigns contiguous angular slices to each truck. Each slice is then routed internally. Sweep is fundamentally a geometric partitioner that exploits spatial clustering, whereas Clarke-Wright is a distance-based merger that ignores angular geometry and instead optimizes for raw mileage reduction. Sweep would produce radial wedge-shaped routes rather than the chain-like routes Clarke-Wright creates, and it may underperform when constraints force non-geographic groupings (for example, "truck 2 only" packages located at opposite ends of the delivery area).

Data Structure Design and Verification

Overview of the Chosen Data Structure

Package records are stored in a **chaining hash table** implemented entirely with Python lists. No built-in dictionaries, sets, or third-party libraries are used. The table maps each package's unique integer ID to a bucket that contains the full composite record: delivery address, delivery deadline, delivery city, delivery ZIP code, package weight, and delivery status (including the delivery time when applicable).

How the Hash Table Satisfies Scenario Requirements

Insertion takes the package ID as input and stores the complete record. Lookup takes the same package ID and returns each required field. Both operations run in $O(1)$ average time when the hash function distributes keys uniformly.

Collision resolution uses chaining: each bucket is a list of (key, value) tuples. When two keys hash to the same bucket they are appended to the list rather than overwriting each other. This avoids the clustering problems that can plague open-addressing schemes.

The table is also self-adjusting. When the load factor exceeds 0.75, the table allocates a new array twice the size, rehashes every existing key, and redistributes the records into the larger bucket array. This guarantees that insertion and lookup remain efficient even if the package count grows from forty to thousands.

During the delivery simulation, the hash table is updated in-place. As each truck reaches a destination, the corresponding package's status is overwritten with a string such as "delivered at 10:14 AM" and the delivery timestamp is recorded in the object. Because the record is retrieved by ID in constant time, the supervisor can query any package at any moment and see the exact status and time.

Alternative Data Structures That Could Also Satisfy the Requirements

Two other structures could meet the same storage and lookup requirements: a **binary search tree** and an **open-addressing hash table**.

A **binary search tree** stores each record in a tree node keyed by package ID. Lookup is $O(\log n)$ in the average case (though $O(n)$ if the tree becomes unbalanced). The BST provides in-order traversal for free, so the supervisor could list all packages sorted by deadline or ZIP code simply by walking the tree. The chaining hash table offers no ordering. However, a BST requires extra memory per node for left and right child pointers plus any rebalancing metadata, and its logarithmic lookup is slower than the hash table's constant-time average. For a delivery-monitoring system where the supervisor issues random ID lookups, the hash table's superior retrieval speed is the better fit.

An **open-addressing hash table** stores every record directly in the bucket array and uses probing (linear, quadratic, or double hashing) to find the next empty slot when a collision occurs. This eliminates the pointer overhead of chaining and improves cache locality, which can speed up small-dataset lookups. However, performance degrades sharply as the load factor approaches 1.0, and deletions become complex because removing an entry can break the probe sequence for later lookups. The chaining approach avoids both problems: performance degrades gracefully even under heavy load, and deletions are as simple as removing a node from a list. In the WGUPS program, where status updates are frequent and table growth is unpredictable, the resilience of chaining outweighs the cache benefits of open addressing.

Implementation Details and Testing

The program is implemented in Python 3.12 and executed through the interpreter detected at build time (PyPy preferred for its JIT compiler, with CPython as a fallback). The development environment is defined in a Nix flake (`flake.nix`) that provides a reproducible shell containing Python 3.12, PyPy3, `ruff`, `pyright`, `just`, and `git`. Build tasks are orchestrated through a `justfile`. Source code is formatted and linted with `ruff` and type-checked with `pyright`.

Automated unit tests cover the two data structures built from scratch. `tests/test_hash_table.py` verifies insertion, lookup, key overwrite, collision chaining, automatic resizing, and load-factor maintenance. `tests/test_distance_table.py` verifies symmetric distance lookups, zero diagonal entries, and graceful error handling for unknown addresses. All tests are run with `just test` or `just validate`.

The main execution flow proceeds in seven stages:

1. Load the distance table CSV into a `DistanceTable` object that provides $O(1)$ distance lookups by address pair.
2. Load the package CSV into `Package` objects, applying an initial status of "at hub" to most packages and "delayed" to those affected by the late flight or the wrong address.
3. Insert every `Package` into the custom `HashTable` keyed by `package_id`.
4. Plan routes by parsing `special_notes` for constraints, assigning packages with deadline-first greedy insertion, simulating routes to repair missed deadlines, and running an EOD-only local search to lower mileage.
5. Set departure times: Truck 1 at 8:00 a.m., Truck 2 at 8:00 a.m. (delayed to 9:05 a.m. if carrying late-arrival packages), Truck 3 at 10:30 a.m.
6. Correct package 9's address at 10:20 a.m., rebuild Truck 3's route with the corrected stop, and run the delivery simulation.
7. Enter the interactive CLI, where the supervisor types a time (for example, "8:35 AM") and sees every package's current status, truck assignment, and deadline. Colours indicate status: green for delivered, yellow for en route, red for delayed, and blue for at hub.

Screenshots Required for Evidence

The following four screenshots must be captured from the running program and submitted as supporting documentation.

Status Check at 8:35 a.m. — 9:25 a.m.

Run the program and enter 8:35 AM at the prompt. The output will show all forty packages on their assigned trucks. The evaluator should observe packages 6, 9, 25, 28, and 32 in "delayed" status; packages 2, 7, 14, 15, 16, 29, 33, and 34 already delivered; Truck 1 and Truck 2 packages en route; and Truck 3 packages still at the hub because that truck has not yet departed.

```
bradwhite 2:55 PM just run-compiled
WGUPS Routing Program
Total mileage for all trucks: 89.3 miles
Enter a time (e.g. '8:35 AM', '12:03 PM', '1:12 PM') or 'all' for final status, or 'q' to quit: 8:35 AM
Status at 8:35 AM:
Package 1: at hub (Truck 2, Deadline: 10:30:00)
Package 2: delivered at 8:34 AM (Truck 1, Deadline: EOD)
Package 3: at hub (Truck 2, Deadline: EOD)
Package 4: at hub (Truck 2, Deadline: EOD)
Package 5: at hub (Truck 3, Deadline: EOD)
Package 6: delayed (Truck 2, Deadline: 10:30:00)
Package 7: delivered at 8:29 AM (Truck 1, Deadline: EOD)
Package 8: at hub (Truck 3, Deadline: EOD)
Package 9: delayed (Truck 3, Deadline: EOD)
Package 10: at hub (Truck 2, Deadline: EOD)
Package 11: at hub (Truck 2, Deadline: EOD)
Package 12: en route (Truck 1, Deadline: EOD)
Package 13: en route (Truck 1, Deadline: 10:30:00)
Package 14: delivered at 8:18 AM (Truck 1, Deadline: 10:30:00)
Package 15: delivered at 8:11 AM (Truck 1, Deadline: 09:00:00)
Package 16: delivered at 8:11 AM (Truck 1, Deadline: 10:30:00)
Package 17: at hub (Truck 2, Deadline: EOD)
Package 18: at hub (Truck 2, Deadline: EOD)
Package 19: en route (Truck 1, Deadline: EOD)
Package 20: en route (Truck 1, Deadline: 10:30:00)
Package 21: en route (Truck 1, Deadline: EOD)
Package 22: at hub (Truck 3, Deadline: EOD)
Package 23: at hub (Truck 2, Deadline: EOD)
Package 24: at hub (Truck 3, Deadline: EOD)
Package 25: delayed (Truck 2, Deadline: 10:30:00)
Package 26: at hub (Truck 3, Deadline: EOD)
Package 27: at hub (Truck 3, Deadline: EOD)
Package 28: delayed (Truck 2, Deadline: EOD)
Package 29: delivered at 8:29 AM (Truck 1, Deadline: 10:30:00)
Package 30: at hub (Truck 2, Deadline: 10:30:00)
Package 31: en route (Truck 1, Deadline: 10:30:00)
Package 32: delayed (Truck 2, Deadline: EOD)
Package 33: delivered at 8:34 AM (Truck 1, Deadline: EOD)
Package 34: delivered at 8:11 AM (Truck 1, Deadline: 10:30:00)
Package 35: at hub (Truck 3, Deadline: EOD)
Package 36: at hub (Truck 2, Deadline: EOD)
Package 37: at hub (Truck 2, Deadline: 10:30:00)
Package 38: at hub (Truck 2, Deadline: EOD)
Package 39: en route (Truck 1, Deadline: EOD)
Package 40: en route (Truck 1, Deadline: 10:30:00)
Enter a time (e.g. '8:35 AM', '12:03 PM', '1:12 PM') or 'all' for final status, or 'q' to quit:
```

Status Check at 9:35 a.m. — 10:25 a.m.

Run the program and enter 9:10 AM. The evaluator should observe package 9 still "delayed" because its address is not corrected until 10:20 a.m.; packages 6, 25, 28, and 32 now "en route" or already delivered (Truck 2 departed at 9:05 a.m.); and Truck 3 packages "at hub" pending the 10:30 a.m. departure.

```
Enter a time (e.g. '8:35 AM', '12:03 PM', '1:12 PM') or 'all' for final status, or 'q' to quit: 9:35 AM
```

Status at 9:35 AM:

```
Package 1: en route           (Truck 2, Deadline: 10:30:00)
Package 2: delivered at 8:34 AM (Truck 1, Deadline: EOD)
Package 3: en route           (Truck 2, Deadline: EOD)
Package 4: en route           (Truck 2, Deadline: EOD)
Package 5: at hub             (Truck 3, Deadline: EOD)
Package 6: en route           (Truck 2, Deadline: 10:30:00)
Package 7: delivered at 8:29 AM (Truck 1, Deadline: EOD)
Package 8: at hub             (Truck 3, Deadline: EOD)
Package 9: delayed            (Truck 3, Deadline: EOD)
Package 10: en route          (Truck 2, Deadline: EOD)
Package 11: delivered at 9:29 AM (Truck 2, Deadline: EOD)
Package 12: delivered at 9:29 AM (Truck 1, Deadline: EOD)
Package 13: delivered at 9:05 AM (Truck 1, Deadline: 10:30:00)
Package 14: delivered at 8:18 AM (Truck 1, Deadline: 10:30:00)
Package 15: delivered at 8:11 AM (Truck 1, Deadline: 09:00:00)
Package 16: delivered at 8:11 AM (Truck 1, Deadline: 10:30:00)
Package 17: en route          (Truck 2, Deadline: EOD)
Package 18: delivered at 9:33 AM (Truck 2, Deadline: EOD)
Package 19: delivered at 9:34 AM (Truck 1, Deadline: EOD)
Package 20: en route          (Truck 1, Deadline: 10:30:00)
Package 21: en route          (Truck 1, Deadline: EOD)
Package 22: at hub            (Truck 3, Deadline: EOD)
Package 23: delivered at 9:31 AM (Truck 2, Deadline: EOD)
Package 24: at hub            (Truck 3, Deadline: EOD)
Package 25: delivered at 9:13 AM (Truck 2, Deadline: 10:30:00)
Package 26: at hub            (Truck 3, Deadline: EOD)
Package 27: at hub            (Truck 3, Deadline: EOD)
Package 28: en route          (Truck 2, Deadline: EOD)
Package 29: delivered at 8:29 AM (Truck 1, Deadline: 10:30:00)
Package 30: en route          (Truck 2, Deadline: 10:30:00)
Package 31: delivered at 8:46 AM (Truck 1, Deadline: 10:30:00)
Package 32: en route          (Truck 2, Deadline: EOD)
Package 33: delivered at 8:34 AM (Truck 1, Deadline: EOD)
Package 34: delivered at 8:11 AM (Truck 1, Deadline: 10:30:00)
Package 35: at hub            (Truck 3, Deadline: EOD)
Package 36: en route          (Truck 2, Deadline: EOD)
Package 37: en route          (Truck 2, Deadline: 10:30:00)
Package 38: en route          (Truck 2, Deadline: EOD)
Package 39: delivered at 9:05 AM (Truck 1, Deadline: EOD)
Package 40: delivered at 8:40 AM (Truck 1, Deadline: 10:30:00)
```

```
Enter a time (e.g. '8:35 AM', '12:03 PM', '1:12 PM') or 'all' for final status, or 'q' to quit: █
```

Status Check at 12:03 p.m. — 1:12 p.m.

Run the program and enter 12:30 PM. All forty packages should show "delivered at <time>" with no pending deliveries. Truck 3's last deliveries occur around 11:10 a.m.–11:46 a.m., confirming that the day ends only when every package has been served.

```
Enter a time (e.g. '8:35 AM', '12:03 PM', '1:12 PM') or 'all' for final status, or 'q' to quit: 12:03 PM
```

Status at 12:03 PM:

```
Package 1: delivered at 10:09 AM (Truck 2, Deadline: 10:30:00)
Package 2: delivered at 8:34 AM (Truck 1, Deadline: EOD)
Package 3: delivered at 10:26 AM (Truck 2, Deadline: EOD)
Package 4: delivered at 10:03 AM (Truck 2, Deadline: EOD)
Package 5: delivered at 11:10 AM (Truck 3, Deadline: EOD)
Package 6: delivered at 9:51 AM (Truck 2, Deadline: 10:30:00)
Package 7: delivered at 8:29 AM (Truck 1, Deadline: EOD)
Package 8: delivered at 11:10 AM (Truck 3, Deadline: EOD)
Package 9: delivered at 11:13 AM (Truck 3, Deadline: EOD)
Package 10: delivered at 10:36 AM (Truck 2, Deadline: EOD)
Package 11: delivered at 9:29 AM (Truck 2, Deadline: EOD)
Package 12: delivered at 9:29 AM (Truck 1, Deadline: EOD)
Package 13: delivered at 9:05 AM (Truck 1, Deadline: 10:30:00)
Package 14: delivered at 8:18 AM (Truck 1, Deadline: 10:30:00)
Package 15: delivered at 8:11 AM (Truck 1, Deadline: 09:00:00)
Package 16: delivered at 8:11 AM (Truck 1, Deadline: 10:30:00)
Package 17: delivered at 9:56 AM (Truck 2, Deadline: EOD)
Package 18: delivered at 9:33 AM (Truck 2, Deadline: EOD)
Package 19: delivered at 9:34 AM (Truck 1, Deadline: EOD)
Package 20: delivered at 9:36 AM (Truck 1, Deadline: 10:30:00)
Package 21: delivered at 9:36 AM (Truck 1, Deadline: EOD)
Package 22: delivered at 11:46 AM (Truck 3, Deadline: EOD)
Package 23: delivered at 9:31 AM (Truck 2, Deadline: EOD)
Package 24: delivered at 11:36 AM (Truck 3, Deadline: EOD)
Package 25: delivered at 9:13 AM (Truck 2, Deadline: 10:30:00)
Package 26: delivered at 11:42 AM (Truck 3, Deadline: EOD)
Package 27: delivered at 10:54 AM (Truck 3, Deadline: EOD)
Package 28: delivered at 10:07 AM (Truck 2, Deadline: EOD)
Package 29: delivered at 8:29 AM (Truck 1, Deadline: 10:30:00)
Package 30: delivered at 10:24 AM (Truck 2, Deadline: 10:30:00)
Package 31: delivered at 8:46 AM (Truck 1, Deadline: 10:30:00)
Package 32: delivered at 9:58 AM (Truck 2, Deadline: EOD)
Package 33: delivered at 8:34 AM (Truck 1, Deadline: EOD)
Package 34: delivered at 8:11 AM (Truck 1, Deadline: 10:30:00)
Package 35: delivered at 10:54 AM (Truck 3, Deadline: EOD)
Package 36: delivered at 9:46 AM (Truck 2, Deadline: EOD)
Package 37: delivered at 10:24 AM (Truck 2, Deadline: 10:30:00)
Package 38: delivered at 10:24 AM (Truck 2, Deadline: EOD)
Package 39: delivered at 9:05 AM (Truck 1, Deadline: EOD)
Package 40: delivered at 8:40 AM (Truck 1, Deadline: 10:30:00)
```

```
Enter a time (e.g. '8:35 AM', '12:03 PM', '1:12 PM') or 'all' for final status, or 'q' to quit:
```

Total Mileage

Run the program and enter `all` at the prompt. The output header reads Total mileage for all trucks: 89.3 miles, which is below the 140-mile requirement. Every package also shows its final delivered status, truck assignment, and timestamp.

```
Enter a time (e.g. '8:35 AM', '12:03 PM', '1:12 PM') or 'all' for final status, or 'q' to quit: all
Package 1: delivered at 10:09 AM (Truck 2, 195 W Oakland Ave)
Package 2: delivered at 8:34 AM (Truck 1, 2530 S 500 E)
Package 3: delivered at 10:26 AM (Truck 2, 233 Canyon Rd)
Package 4: delivered at 10:03 AM (Truck 2, 380 W 2880 S)
Package 5: delivered at 11:10 AM (Truck 3, 300 State St)
Package 6: delivered at 9:51 AM (Truck 2, 3060 Lester St)
Package 7: delivered at 8:29 AM (Truck 1, 1330 2100 S)
Package 8: delivered at 11:10 AM (Truck 3, 300 State St)
Package 9: delivered at 11:13 AM (Truck 3, 410 S State St)
Package 10: delivered at 10:36 AM (Truck 2, 600 E 900 South)
Package 11: delivered at 9:29 AM (Truck 2, 2600 Taylorsville Blvd)
Package 12: delivered at 9:29 AM (Truck 1, 3575 W Valley Central Station bus Loop)
Package 13: delivered at 9:05 AM (Truck 1, 2010 W 500 S)
Package 14: delivered at 8:18 AM (Truck 1, 4300 S 1300 E)
Package 15: delivered at 8:11 AM (Truck 1, 4580 S 2300 E)
Package 16: delivered at 8:11 AM (Truck 1, 4580 S 2300 E)
Package 17: delivered at 9:56 AM (Truck 2, 3148 S 1100 W)
Package 18: delivered at 9:33 AM (Truck 2, 1488 4800 S)
Package 19: delivered at 9:34 AM (Truck 1, 177 W Price Ave)
Package 20: delivered at 9:36 AM (Truck 1, 3595 Main St)
Package 21: delivered at 9:36 AM (Truck 1, 3595 Main St)
Package 22: delivered at 11:46 AM (Truck 3, 6351 South 900 East)
Package 23: delivered at 9:31 AM (Truck 2, 5100 South 2700 West)
Package 24: delivered at 11:36 AM (Truck 3, 5025 State St)
Package 25: delivered at 9:13 AM (Truck 2, 5383 S 900 East #104)
Package 26: delivered at 11:42 AM (Truck 3, 5383 S 900 East #104)
Package 27: delivered at 10:54 AM (Truck 3, 1060 Dalton Ave S)
Package 28: delivered at 10:07 AM (Truck 2, 2835 Main St)
Package 29: delivered at 8:29 AM (Truck 1, 1330 2100 S)
Package 30: delivered at 10:24 AM (Truck 2, 300 State St)
Package 31: delivered at 8:46 AM (Truck 1, 3365 S 900 W)
Package 32: delivered at 9:58 AM (Truck 2, 3365 S 900 W)
Package 33: delivered at 8:34 AM (Truck 1, 2530 S 500 E)
Package 34: delivered at 8:11 AM (Truck 1, 4580 S 2300 E)
Package 35: delivered at 10:54 AM (Truck 3, 1060 Dalton Ave S)
Package 36: delivered at 9:46 AM (Truck 2, 2300 Parkway Blvd)
Package 37: delivered at 10:24 AM (Truck 2, 300 State St)
Package 38: delivered at 10:24 AM (Truck 2, 300 State St)
Package 39: delivered at 9:05 AM (Truck 1, 2010 W 500 S)
Package 40: delivered at 8:40 AM (Truck 1, 380 W 2880 S)

Total mileage: 89.3 miles
Enter a time (e.g. '8:35 AM', '12:03 PM', '1:12 PM') or 'all' for final status, or 'q' to quit:
```

Future Modifications

If this project were done again, two modifications would improve robustness and maintainability.

First, the deadline-repair logic would be moved into the greedy assignment phase itself. Currently, the program assigns all packages first and then simulates routes to find violations, moving late packages afterward. A more integrated approach would simulate the route **before** committing each new package to a truck, rejecting any insertion that would cause a missed deadline. This would reduce the need for a separate repair loop and would produce better mileage because EOD packages would be placed with full knowledge of which deadline-sensitive packages are already locked in.

Second, the EOD-only local search would be expanded into a full simulated-annealing search that also considers moving 10:30-deadline packages between Trucks 1 and 2. Currently, the local search only moves EOD packages because deadline packages are anchored by their time constraints. However, a 10:30 package on Truck 2 might actually arrive earlier if moved to Truck 1, freeing space on Truck 2 for an EOD package that is currently on Truck 3. A temperature-based search would temporarily accept deadline-package moves that increase cost if they enable later cost-saving EOD swaps, escaping local minima that the current greedy search cannot.

Conclusion

The WGUPS Routing Program demonstrates that a self-adjusting heuristic combined with a custom hash table can solve a real-world vehicle-routing problem efficiently, correctly, and within tight constraints.

The deadline-aware assignment pipeline, Clarke-Wright savings algorithm, and 2-opt local search deliver all forty packages in 89.3 miles — well under the 140-mile ceiling — while honoring every truck restriction, deadline, and mid-flight address correction. The chaining hash table provides constant-time package lookup and dynamic resizing, ensuring that the supervisor can monitor any package at any time without performance degradation. Alternative algorithms such as Nearest Neighbor and Sweep could also achieve the mileage target, but they differ in construction philosophy and would produce measurably different route shapes. Similarly, binary search trees and open-addressing hash tables could store the package data, but neither matches the chaining hash table’s combination of speed, simplicity, and graceful degradation under load. Future iterations should integrate deadline simulation directly into the greedy assignment and expand the local search to consider time-sensitive package moves for further mileage reduction.

References

Vehicle routing problem. (n.d.). In **Wikipedia**. https://en.wikipedia.org/wiki/Vehicle_routing_problem

2-opt. (n.d.). In **Wikipedia**. <https://en.wikipedia.org/wiki/2-opt>

Hash table. (n.d.). In **Wikipedia**. https://en.wikipedia.org/wiki/Hash_table